Group Presentations

Introduction to Machine Learning Theory, M2 BDIA
Nahla Ben Amor
Friday 16 October

Each group presents for 30 minutes (maximum 35 slides) and writes a short Python program on the Bank Marketing dataset (data from a bank: for each client, we know whether he subscribed to a term deposit after a phone call).

For each topic, the plan and the program below are only proposals. You can change them if you want. The Python program should be presented briefly at the end of the presentation.

Topic 1: Evolutionary Algorithms in Machine Learning

Group: to be formed.

In this presentation, the objective is to explain how evolutionary algorithms imitate natural evolution to find a good solution among a huge number of possibilities, and how they are used in machine learning.

Proposed plan:

1. The principle of evolutionary algorithms.
2. How a genetic algorithm works, on a small example.
3. Uses of genetic algorithms in machine learning.
4. Limits of the method.
5. Your program and its results.

Proposed Python program: write a genetic algorithm yourselves, without a ready-made library, that chooses which variables to give to a decision tree. Split the data into three parts: training, validation and test. Each candidate is a list of variables, and its score is the quality of the tree measured on the validation part. Keep the best candidate from one generation to the next, so that the best score never goes down. Show how the best score evolves over the generations, and compare with a tree that uses all the variables, trained and tested on the same parts (quality on the test part and number of variables used).

Data: the file `bank-additional-full.csv` (`https://archive.ics.uci.edu/dataset/222/bank+marketing`). We want to predict, just before a call, whether the client will subscribe, so use only information known at that moment: remove the variable `duration` (length of the call). The variable `campaign` (number of calls to this client in this campaign, including this one) is known at that moment. Only 11% of the clients subscribe, so measure the quality with the balanced accuracy, not the accuracy.

Proposed reference: T. Mitchell, *Machine Learning*, 1997, chapter 9 "Genetic Algorithms" (free PDF of the book): `https://www.cs.cmu.edu/~tom/mlbook.html`

Topic 2: Agentic AI

Group: to be formed.

In this presentation, the objective is to explain what an AI agent is (a system that observes its environment, decides and acts to reach a goal), how modern agentic AI systems can operate with one or several agents, and how language models like ChatGPT are used today to build such systems.

Proposed plan:

1. What an AI agent is, and the classical types of agents.
2. From AI agents to agentic AI, with its new capabilities and architectures.

3. Agentic systems built on language models, and how they differ from a chatbot.
4. Risks and limits of agentic AI.
5. Your program and its results.

Proposed Python program: write two small Python functions (for example one that computes a statistic on the data and one that trains a model) and an agent, built on a language model, that uses them to answer two questions. The agent itself must decide which function to call and with which arguments. Show each step taken by the agent, and one example where it gets it wrong. You may use a free model (Ollama) or a free online service.

Data: the file `bank-additional-full.csv` (`https://archive.ics.uci.edu/dataset/222/bank+marketing`). If your agent trains a model to predict, just before a call, whether the client will subscribe, it must use only information known at that moment: remove the variable `duration` (length of the call).

Proposed references: S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th edition, 2020, chapter 2 "Intelligent Agents". Anthropic, *Building Effective Agents*, 2024: `https://www.anthropic.com/research/building-effective-agents`

Topic 3: Case-Based Reasoning and Retrieval-Augmented Generation

Group: Adam Fatnassi, Ameni Ben Youssef, Ranim Mhiri, Lina Pioche.

In this presentation, the objective is to explain how to solve a new problem by reusing the solutions of the most similar past problems, like a doctor who remembers similar patients, and to link this idea with Retrieval-Augmented Generation.

Proposed plan:

1. The principle of case-based reasoning and its cycle.
2. Similarity between cases, and the difference with the k-nearest neighbours method.
3. Retrieval-Augmented Generation and its link with case-based reasoning.
4. Limits of case-based reasoning.
5. Your program and its results.

Proposed Python program: build a simple version of the case-based reasoning cycle, not only a classification by neighbours. For a new client, (1) find the most similar past clients, with a similarity that you define yourselves (explain the weights you give to the variables); (2) reuse their outcome to predict whether he will subscribe, and explain the prediction with the clients found; (3) once the real outcome is known, check whether the prediction was right and, if not, correct it; (4) add the new client, with his real outcome, to the case base. First compare your system with the k-nearest neighbours method of scikit-learn on the same test clients and the same case base, without adding new cases. Then, in a separate experiment, take the test clients one by one: predict first, and only then add the client to the case base. Show how the quality changes, separately from the results of the first comparison.

Data: the file `bank-additional-full.csv` (`https://archive.ics.uci.edu/dataset/222/bank+marketing`). We want to predict, just before a call, whether the client will subscribe, so use only information known at that moment: remove the variable `duration` (length of the call). The variable `campaign` (number of calls to this client in this campaign, including this one) is known at that moment. Only 11% of the clients subscribe, so measure the quality with the balanced accuracy, not the accuracy.

Proposed reference: T. Mitchell, *Machine Learning*, 1997, chapter 8 "Instance-Based Learning", with section 8.5 on case-based reasoning (free PDF of the book): `https://www.cs.cmu.edu/~tom/mlbook.html`

Topic 4: Explainable AI (XAI)

Group: Feriel Bouguecha, Cyrine Fakhfakh, Wifek Chabir, Zakary Hammoutou, Soumaya Rodmhani.

In this presentation, the objective is to explain how we can understand why a model takes a decision, so that we can trust it, correct it and justify it.

Proposed plan:

1. Why explain a model.
2. Models that are easy to understand.
3. Methods to explain a complex model.
4. Limits of these explanations.
5. Your program and its results.

Proposed Python program: train a complex model (gradient boosting) and a small decision tree, and compare their quality on test clients that were not used for training. For the complex model, show which variables matter most, explain the decision for one client, and look for a realistic change in a variable controlled by the bank (for example the contact channel) that would change the prediction of the model. If no realistic change works, say so and explain why.

Data: the file `bank-additional-full.csv` (`https://archive.ics.uci.edu/dataset/222/bank+marketing`). We want to predict, just before a call, whether the client will subscribe, so use only information known at that moment: remove the variable `duration` (length of the call). The variable `campaign` (number of calls to this client in this campaign, including this one) is known at that moment. Only 11% of the clients subscribe, so measure the quality with the balanced accuracy, not the accuracy.

Proposed reference: C. Molnar, *Interpretable Machine Learning* (free online book): `https://christophm.github.io/interpretable-ml-book/`